

Level 2 – Intermediate

Duração estimada: 4 meses (16 semanas · 5-10h/semana)

Pré-requisito: Nível 1 completo (ou conhecimento equivalente)

Foco: Tempo real, caching, arquitetura limpa, NoSQL, escalabilidade

Projetos: URL Shortener · Chat Real-Time · E-commerce API



Skills ao final do nível

- [] **WebSockets + SSE:** Socket.io, rooms, Redis adapter, sticky sessions, quando usar WS vs SSE
 - [] **Redis:** Cache-aside, TTL, invalidação, pub/sub, rate limiting, filas simples
 - [] **MongoDB:** Schema design para documentos, aggregation pipeline, índices
 - [] **Nginx:** Reverse proxy, load balancing, SSL, compressão (gzip/brotli), HTTP caching (Cache-Control/ETag), CORS
 - [] **HTTP Caching:** Cache-Control, ETag, Last-Modified, 304 Not Modified, estratégias por tipo de recurso
 - [] **Compressão:** gzip vs brotli, quando comprimir, trade-off CPU vs banda
 - [] **CORS:** Preflight, headers, quando o browser envia OPTIONS
 - [] **Clean Architecture:** Camadas (domain, application, infrastructure), DI, use cases
 - [] **Design Patterns:** Repository, Factory, Adapter, Observer – quando aplicar
 - [] **Testes:** Integração com banco real, contract tests
 - [] **CI/CD avançado:** Múltiplos estágios, services no pipeline, Docker push
 - [] **Algoritmos:** BFS/DFS, árvores, pilha, fila, hash tables, two pointers
 - [] **Performance:** N+1 queries, lazy loading, eager loading, índices compostos
 - [] **Idempotency Key:** Evitar duplicidade em operações críticas (pagamentos)
 - [] **Backpressure:** Lidar com consumidores lentos, circuit breaker
-



Semana a Semana

SEMANA 1 – WebSockets + SSE: Comunicação em Tempo Real

📖 **Teoria (4h)** - Leia: [Socket.io Docs – Rooms, Namespaces](#) - Assista: [Hussein Nasser – WebSockets vs HTTP](#) - Estude: handshake WS, diferenças para polling, sticky sessions

SSE (Server-Sent Events) – alternativa mais simples:

Quando usar WebSocket vs SSE?

- WebSocket: comunicação bidirecional (chat, jogos, colaboração)
- SSE: apenas notificações do servidor (feed, alerts, status updates)

SSE é mais simples:

- Usa HTTP puro (text/event-stream)
- Reconexão automática (Last-Event-ID)
- Funciona com HTTP/1.1 sem upgrades
- Mas: só unidirecional (servidor → cliente), limite de conexões por domínio

```
// SSE endpoint (Express)
app.get('/api/events/stream', (req, res) => {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive'
  });

  // Envia eventos a cada 5s
  const interval = setInterval(() => {
    res.write(`data: ${JSON.stringify({ timestamp: Date.now() })}\n\n`);
  }, 5000);

  req.on('close', () => clearInterval(interval));
});

// Cliente (browser)
const source = new EventSource('/api/events/stream');
source.onmessage = (event) => console.log(JSON.parse(event.data));
```



Prática (5h)

```
import { Server } from 'socket.io';
import http from 'http';

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' },
  adapter: createRedisAdapter(redisClient) // para escalar
});

io.on('connection', (socket) => {
  // Rooms
  socket.on('join:room', (room) => {
    socket.join(room);
    socket.to(room).emit('user:joined', socket.id);
  });

  // Mensagens
  socket.on('message:send', ({ room, content }) => {
    io.to(room).emit('message:received', {
      userId: socket.data.userId,
      content,
      timestamp: new Date()
    });
  });

  // Typing indicator
  socket.on('typing:start', (room) => {
    socket.to(room).emit('typing:update', socket.data.userId);
  });
});
```

 **Exercício:** Implemente 3 salas de chat simultâneas, com mensagens privadas (socket-to-socket) via `socket.to(id).emit()`.

SEMANA 2 – Redis: Padrões de Cache + Pub/Sub

 **Teoria (3h)** - Leia: [Redis Patterns \(cache-aside, TTL, invalidação\)](#) - Assista: [Redis Crash Course \(Fireship\)](#) - Estude: cache stampede, write-through, write-behind

 **Prática (5h)**

```
import Redis from 'ioredis';

const redis = new Redis();

// Cache-aside pattern
async function getOrSet<T>({
  key: string,
  fetch: () => Promise<T>,
  ttl = 60
}): Promise<T> {
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const data = await fetch();
  await redis.set(key, JSON.stringify(data), 'EX', ttl);
  return data;
}

// Invalidação: quando dados mudam, deletar cache
await redis.del(`product:${productId}`);

// Pub/Sub para escalar WebSockets
const pub = new Redis();
const sub = new Redis();
sub.subscribe('chat:events');
sub.on('message', (channel, msg) => {
  // reenviar para todos os sockets NESTA instância
  io.to(JSON.parse(msg).room).emit('message:received', JSON.parse(msg));
});

// Rate limiting
async function rateLimit(ip: string, max = 10, window = 60): Promise<boolean> {
  const key = `rate:${ip}`;
  const current = await redis.incr(key);
  if (current === 1) await redis.expire(key, window);
  return current <= max;
}
```

 **Exercício:** Implemente rate limiting por IP + por rota (ex: /api/auth permite 5 req/min, /api/products 60 req/min).

SEMANA 3 – Mini Projeto: URL Shortener (Cache + Redirect)

Stack: Node/TS + Redis + Docker

Conceitos aplicados: cache-aside, Base62 encoding, HTTP redirects, contadores atômicos

```
POST /shorten { "url": "https://..." }
→ { "shortUrl": "http://localhost:3000/aB3xK9m" }

GET /aB3xK9m
→ 302 Redirect para URL original

GET /aB3xK9m/stats
→ { "shortId": "aB3xK9m", "clicks": 42, "createdAt": "..." }
```

Checklist: - [] Geração de hash único (Base62, 7 chars) - [] Cache Redis com TTL (URL → shortId, shortId → URL) - [] Incremento atômico de cliques (**INCR**) - [] Validação de URL - [] Docker + docker-compose - [] Testes

SEMANA 4 – MongoDB: Modelagem e Aggregation

 **Teoria (3h)** - Leia: [MongoDB – Schema Design](#) - Leia: [MongoDB – Aggregation Pipeline](#) - Estude: embedded vs referenced documents, quando usar MongoDB vs PostgreSQL

Prática (5h)

```
// Modelagem: mensagens de chat
const messageSchema = new Schema({
  roomId: { type: String, required: true, index: true },
  userId: { type: String, required: true },
  content: { type: String, required: true },
  createdAt: { type: Date, default: Date.now, index: true }
});

// Índice composto para queries de histórico
messageSchema.index({ roomId: 1, createdAt: -1 });

// Aggregation: últimas 50 msg com dados do usuário
const messages = await Message.aggregate([
  { $match: { roomId } },
  { $sort: { createdAt: -1 } },
  { $limit: 50 },
  { $lookup: {
    from: 'users',
    localField: 'userId',
    foreignField: '_id',
    as: 'user'
  } },
  { $unwind: '$user' },
  { $project: { content: 1, createdAt: 1, 'user.name': 1, 'user.avatar': 1 } }
]);
```

 **Exercício:** Modele o esquema de um chat real (rooms, messages, users) e crie 3 pipelines de agregação.

SEMANA 5 – Nginx: HTTP Caching, Compressão, Proxy e SSL

📖 **Teoria (4h)** - Leia: [Nginx – Reverse Proxy](#) - Leia: [HTTP Caching \(MDN\)](#) - Assista: [Nginx Crash Course \(Fireship\)](#)

HTTP Caching (fundamental para performance):

Cache-Control headers:

- |— max-age=3600 → cache por 1h no browser
- |— s-maxage=3600 → cache em CDN/proxy
- |— no-cache → revalidar com servidor (ETag)
- |— no-store → nunca armazenar (dados sensíveis)
- |— public vs private → proxy cache? ou só browser?
- |— immutable → recurso nunca muda (js/css versão hash)

ETag / If-None-Match:

- Servidor envia ETag (hash do conteúdo)
- Cliente envia If-None-Match na próxima request
- Se recurso não mudou → 304 Not Modified (sem body!)

Last-Modified / If-Modified-Since:

- Alternativa ao ETag (data de modificação)

Estratégias de cache:

- |— Static assets: max-age=31536000, immutable (versão hash no filename)
- |— API responses: ETag + no-cache (revalida)
- |— HTML pages: no-cache (sempre fresh)
- |— Dados sensíveis: no-store

Compressão (gzip, brotli):

Por que comprimir?

- |— JSON: reduz ~70-80% do tamanho
- |— HTML: reduz ~60-70%
- |— Custo: CPU (mínimo) vs banda (caro)

Brotli (mais eficiente) vs gzip (mais compatível):

- |— Brotli: ~20% melhor compressão que gzip
- |— gzip: suporte universal desde HTTP/1.0
- |— Melhor prática: brotli com fallback gzip

CORS (Cross-Origin Resource Sharing):

Por que existe? → Segurança do browser (mesma origem)

Headers de resposta:

- |— Access-Control-Allow-Origin → domínios permitidos
- |— Access-Control-Allow-Methods → GET, POST, PUT, DELETE, OPTIONS
- |— Access-Control-Allow-Headers → Content-Type, Authorization
- |— Access-Control-Expose-Headers → headers visíveis ao JS
- |— Access-Control-Max-Age → cache de preflight

Preflight (OPTIONS): quando o browser envia?

- |— Método diferente de GET/HEAD/POST
- |— Content-Type diferente de form-urlencoded
- |— Headers customizados

 **Prática (5h)**

```

http {
    # Compressão gzip
    gzip on;
    gzip_types application/json text/plain text/css application/javascript;
    gzip_min_length 1000;
    gzip_vary on;

    # Brotli (se compilado com módulo)
    brotli on;
    brotli_types application/json text/plain text/css application/javascript;

    upstream backend {
        server app1:3001;
        server app2:3002;
        ip_hash; # sticky sessions p/ WebSocket
    }

    server {
        listen 80;
        return 301 https://$host$request_uri;
    }

    server {
        listen 443 ssl http2;
        ssl_certificate /etc/nginx/ssl/cert.pem;
        ssl_certificate_key /etc/nginx/ssl/key.pem;

        # CORS headers (adicional ao que a API envia)
        add_header Access-Control-Allow-Origin "https://meuapp.com" always;
        add_header Access-Control-Allow-Methods "GET, POST, PUT, DELETE, OPTIONS" always;
        add_header Access-Control-Allow-Headers "Content-Type, Authorization" always;

        # Rate limiting
        limit_req zone=api limit=30r/s burst=10;

        location / {
            proxy_pass http://backend;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade"; # WebSocket support
            proxy_set_header Host $host;
        }

        # Cache de estáticos (1 ano, imutável)
        location /static/ {
            expires 31536000;
            add_header Cache-Control "public, immutable";
        }

        # API com ETag (pass-through)
        location /api/ {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            # Não armazenar em cache proxy, deixar ETag do backend
            proxy_no_cache $http_pragma $http_authorization;
        }
    }

    limit_req_zone $binary_remote_addr zone=api:10m rate=30r/s;
}

```


 **Exercício:** Configure Nginx com: - Proxy reverso para 2 instâncias do URL Shortener - SSL auto-assinado + HTTP/2 - Compressão gzip + brotli - CORS configurado - Cache de estáticos (1 ano) - Rate limiting (30 req/s) - Teste com curl verificando headers de cache e compressão

SEMANA 6 – Clean Architecture + Design Patterns (Profundo)

 **Teoria (5h)** - Leia: **Padrões de Arquitetura** (Eduardo S. Rocha) – caps sobre Clean Architecture - Leia: [Refactoring Guru – Design Patterns](#) (Factory, Adapter, Repository, Observer) - Assista: [Erick Wendel – Clean Architecture no Node](#)

Prática (5h)

```
// Estrutura de pastas (Clean Architecture)
src/
├── domain/
│   ├── entities/           → regras PURAS de negócio
│   │   └── Task.ts         (validações, mudanças de estado)
│   ├── use-cases/         → orquestração
│   │   └── CreateTask.ts
│   └── application/
│       ├── ports/          → interfaces (contratos)
│       │   ├── ITaskRepository.ts
│       │   └── IEmailService.ts
│       └── services/        → lógica de aplicação
│           └── PostgresTaskRepository.ts → implements ITaskRepository
├── infrastructure/
│   ├── database/
│   │   └── PostgresTaskRepository.ts → implements ITaskRepository
│   ├── http/
│   │   └── TaskController.ts
│   ├── email/
│   │   └── SendGridService.ts
│   └── main/
│       ├── container.ts     → DI (inversão de dependência)
│       └── server.ts
└──
```

// REGRA DE OURO: domain/ NÃO importa nada de infrastructure/
// Infrastructure DEPENDE de domain, nunca o contrário

 **Exercício:** Migre a Task Manager do nível 1 para Clean Architecture. Extraia interfaces para repositórios e use DI.

SEMANAS 7-8 – Projeto: Chat Real-Time

Stack: Node/TS + Socket.io + MongoDB + Redis + Nginx
Arquitetura: Clean Architecture

Funcionalidades: - Salas de chat com join/leave - Mensagens em tempo real (WebSocket) - Histórico (MongoDB) - Redis pub/sub para escalar múltiplas instâncias - Status online/offline (chave com TTL no Redis) - Nginx com sticky sessions + SSL - Upload de imagens (multer)

Conceitos aplicados: | Conceito | Implementação |
|-----|-----| | WebSocket rooms | `socket.join(room)` / `io.to(room).emit()` | | Escalabilidade | Redis adapter do Socket.io | | Persistência | MongoDB (mensagens) | | Cache + status | Redis (TTL para online) | | Proxy reverso | Nginx com sticky sessions | | Clean Architecture | Use cases, repositories, DI |

Checklist: - [] Autenticação via JWT - [] Salas dinâmicas (criar/entrar/sair) - [] Histórico com paginação - [] Redis pub/sub (testar com 2 instâncias) - [] Upload de imagens - [] Nginx configurado - [] Docker compose (app x2, mongo, redis, nginx) - [] Clean Architecture

SEMANA 9 – Testes de Integração

📖 **Teoria (2h)** - Leia: [Testcontainers for Node.js](#) - Estude: quando testar integração vs unitário, pirâmide de testes

🔧 **Prática (5h)**

```
import { GenericContainer } from 'testcontainers';
import { MongoClient } from 'mongodb';

describe('MessageRepository (integração)', () => {
  let container;
  let client: MongoClient;
  let repo: MessageRepository;

  beforeAll(async () => {
    container = await new GenericContainer('mongo:7')
      .withExposedPorts(27017)
      .start();
    const url = `mongodb://localhost:${container.getMappedPort(27017)}`;
    client = new MongoClient(url);
    await client.connect();
    repo = new MessageRepository(client.db('test'));
  }, 30000);

  afterAll(async () => {
    await client.close();
    await container.stop();
  });

  it('should save and retrieve messages', async () => {
    await repo.save({ room: 'test', content: 'Hello', userId: '1' });
    const messages = await repo.findByRoom('test');
    expect(messages).toHaveLength(1);
    expect(messages[0].content).toBe('Hello');
  });
});
```

 **Exercício:** Escreva 3 testes de integração para o Chat (salvar msg, buscar histórico, listar salas).

SEMANA 10 – CI/CD Avançado + Deploy

 **Teoria (2h)** - GitHub Actions: matrix builds, services, caching, artifacts

 **Prática (5h)**

```

name: CI/CD Chat

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18, 20]
    services:
      mongodb:
        image: mongo:7
        ports: [27017:27017]
      redis:
        image: redis:7-alpine
        ports: [6379:6379]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: ${{ matrix.node-version }} }
      - run: npm ci
      - run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push Docker
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: seuuser/chat-app:${{ github.sha }}
      - name: Deploy to server
        run: |
          ssh user@server "docker pull seuuser/chat-app:${{ github.sha }} && docker compose up"

```

SEMANA 11 – Algoritmos + Estruturas de Dados

 **Teoria (4h)** - Leia: **Entendendo Algoritmos** – Caps 5 a 10 (hash tables, BFS, DFS, Dijkstra, greedy) - Estude: pilha, fila, árvore binária, grafo, hash table

 **Prática (5h)**

```
// Implemente do zero:
class Stack<T> { ... }           // push, pop, peek
class Queue<T> { ... }          // enqueue, dequeue
class TreeNode { ... }          // value, left, right

function bfs(root: TreeNode): number[] // 0(n)
function dfs(root: TreeNode): number[] // 0(n)
function isValidBST(root: TreeNode): boolean

// LeetCode (médio):
// - Valid Parentheses (Stack)
// - Binary Tree Level Order Traversal (BFS)
// - Implement Trie (Hash Table + Tree)
```

 **Exercício:** Resolva 8 problemas no LeetCode (nível Easy/Medium).
 Documente a abordagem e complexidade de cada um.

SEMANAS 12-14 – Projeto: E-commerce API

Stack: Node/TS + PostgreSQL + Redis + Nginx + Docker

Arquitetura: Clean Architecture + Repository Pattern

Funcionalidades: | Funcionalidade | Implementação |
 |-----|-----| | CRUD Produtos + Categorias | REST +
 validação Zod | | Autenticação c/ roles | JWT + middleware
 requireRole('admin') | | Carrinho | Sessão no Redis ou tabela no
 PostgreSQL | | Checkout | Transação ACID (BEGIN/COMMIT/ROLLBACK) | |
 Cache | Redis (produtos populares, categorias) | | Nginx | Reverse proxy
 + rate limiting + SSL | | CI/CD | Pipeline completa com deploy |

Idempotency Key (padrão crítico para APIs financeiras):

```
// Cliente envia: POST /api/checkout com header Idempotency-Key: uuid-unico
// Servidor:
// 1. Verifica se já processou essa key (Redis: set NX com TTL 24h)
// 2. Se já processou → retorna resultado anterior (evita duplicidade)
// 3. Se não → processa normalmente

async function checkout(idempotencyKey: string, orderData: any) {
  const key = `idempotency:${idempotencyKey}`;

  // Tenta marcar como processando (atomic)
  const alreadyProcessed = await redis.set(key, 'pending', 'EX', 86400, 'NX');
  if (!alreadyProcessed) {
    // Já processado → retorna resultado salvo
    return JSON.parse(await redis.get(`idempotency:result:${idempotencyKey}`));
  }

  try {
    const result = await createOrder(orderData);
    await redis.set(`idempotency:result:${idempotencyKey}`, JSON.stringify(result));
    return result;
  } catch (error) {
    await redis.del(key); // Limpa pra permitir retry
    throw error;
  }
}
```

Backpressure (lidar com consumidores lentos):

Problema: Servidor recebe requests mais rápido que consegue processar

Soluções:

- └─ Reject (503 Service Unavailable) → cliente tenta depois
- └─ Queue com tamanho limitado → descarta se exceder
- └─ Backpressure explícito → http2 RST_STREAM, TCP backpressure
- └─ Circuit Breaker → para de aceitar, recupera gradualmente

No Node: streams com highWaterMark controlável

req.on('data', () => { /* pausa se stream.resume() não aguenta */ });

Checklist adicional: - [] Transação ACID no checkout (estoque + pedido)
 - [] Cache-aside com invalidação (produto atualizado → cache deletado) -
 [] Repository Pattern + interfaces - [] Idempotency key no checkout -
 [] Testes de integração (checkout completo) - [] Nginx configurado
 (compressão + cache + CORS) - [] Docker compose (app + postgres + redis
 + nginx) - [] Swagger

SEMANAS 15-16 – Revisão, LeetCode e Prova

Semana 15: Refatore o E-commerce para Clean Architecture (se não fez).
 Adicione testes de integração. Resolva 5 LeetCode médios.

Semana 16 – Prova de Nível:

1. **WebSocket:** Como escalar para 100k conexões simultâneas?
2. **SSE vs WebSocket:** Quando usar cada um? Cite exemplos
3. **Redis:** Explique cache-aside pattern. Quando ocorre cache stampede e como evitar?
4. **MongoDB:** Quando usar embedded vs referenced documents?
5. **Nginx:** Diferença entre proxy reverso e load balancer. Quando usar cada?
6. **HTTP Caching:** Explique a diferença entre `Cache-Control: no-cache` e `Cache-Control: no-store`
7. **Compressão:** gzip vs brotli – qual mais eficiente? qual trade-off?
8. **CORS:** O que é preflight request? Quando o browser envia OPTIONS?
9. **Clean Architecture:** Por que o domínio não pode depender da infraestrutura? Como a DI resolve isso?
10. **Idempotency:** Por que usar Idempotency-Key em APIs de pagamento?
11. **Algoritmos:** Implemente BFS e DFS em uma árvore binária. Qual a diferença?
12. **CI/CD:** Desenhe uma pipeline com 4 estágios para um sistema que usa banco + Redis.

 **Passa se:** Responde $\geq 5/7$ + projetos Chat e E-commerce completos no GitHub.

Resumo do Nível

Tecnologia	O que você domina
WebSockets	Tempo real, rooms, Redis adapter, sticky sessions
Redis	Cache-aside, pub/sub, rate limiting, TTL
MongoDB	Schema design, aggregation, índices
Nginx	Proxy reverso, LB, SSL, rate limiting, WebSocket support
Clean Architecture	Domain/Application/Infrastructure, DI, use cases
Design Patterns	Repository, Factory, Adapter, Observer
Testes	Integração com banco real (testcontainers)
CI/CD	Pipeline multi-estágio com deploy
Algoritmos	BFS, DFS, Stack, Queue, Tree, Hash Table

Próximo nível: `level-03-advanced.md`